Course Title : HIGH PERFORMANCE COMPUTING Ass-7

Name : J.Varshini

Batch no : 11

Hall Ticket No : 2303A51758

1. Lab: OpenMP SIMD Vectorization Objective:
To understand SIMD vectorization and observe how OpenMP SIMD improves loop execution.
Task 1: Normal Loop Execution
File: simd_serial.c

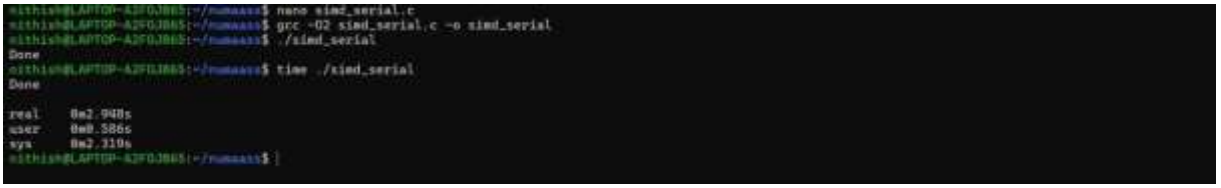
```
#include &lt;stdio.h&gt;

#define N 100000000

int main() { float a[N],
b[N], c[N]; for (int i = 0; i
&lt; N; i++) c[i] = a[i] +
b[i];
printf(&quot;Done\n&quot;
);

06-02-2026 EOD

return 0;
}
```



2. Task 2: OpenMP SIMD Loop

#pragma omp simd

for (int i = 0; i < N;

i++) c[i] = a[i] + b[i];

Compile:

gcc -O2 -fopenmp simd_serial.c -o simd_test

```
  GNU nano 7.2                                numa_first_touch.c *
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 10000000

int main() {
    double *A = (double*)malloc(sizeof(double)*N);

    // First Touch Initialization
    #pragma omp parallel for
    for (int i = 0; i < N; i++)
        A[i] = 1.0;

    double sum = 0;

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < N; i++)
        sum += A[i];

    printf("Sum = %f\n", sum);

    free(A);
    return 0;
}
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo     M-6 Copy       ^Q Where Was
```

```
nithish@LAPTOP-A2FOJ865:~/numaass$ nano simd_serial.c
nithish@LAPTOP-A2FOJ865:~/numaass$ gcc -O2 -fopenmp simd_serial.c -o simd_test
nithish@LAPTOP-A2FOJ865:~/numaass$ time ./simd_test
Done

real    0m3.550s
user    0m0.485s
sys     0m3.058s
nithish@LAPTOP-A2FOJ865:~/numaass$
```

3. Task 3: Conceptual Questions

1. What does SIMD mean?

SIMD stands for Single Instruction, Multiple Data. It means that the processor can perform the same operation on multiple data elements at the same time using a single instruction.

In Task 1 (Normal Loop Execution), the array addition was performed in a normal loop. The CPU processed one element at a time — meaning each addition `c[i] = a[i] + b[i]` was done sequentially.

In Task 2 (OpenMP SIMD Loop), we added `#pragma omp simd` before the loop. This tells the compiler to vectorize the loop. Instead of adding one pair of elements at a time, the CPU can add multiple elements together in parallel using vector registers.

So practically:

- Task 1 → One operation per step (sequential processing)
- Task 2 → Multiple operations per step (vectorized processing using SIMD)

Even though we may not have seen a big timing difference due to memory limitations, SIMD is designed to improve performance by utilizing the processor's vector hardware efficiently.

In simple words, SIMD allows the computer to do the same calculation on many data items at once, making loops faster and more efficient.

2. Does SIMD use multiple threads?

In Task 1 (Normal Loop), one thread processed one element at a time.

- In Task 2 (OpenMP SIMD Loop), we used `#pragma omp simd`, which tells the compiler to vectorize the loop.
    - Still one thread
    - But multiple data elements processed per instruction

So:

- Multithreading (OpenMP parallel for) → Multiple threads working simultaneously.
- SIMD → One thread doing multiple data operations at once.

Simple Difference

- Threads = Parallelism across cores (Thread-level parallelism)
- SIMD = Parallelism inside a core (Data-level parallelism)

So SIMD improves performance without creating extra threads.

3. When is SIMD most effective?

SIMD is most effective when a program performs the same operation repeatedly on a large set of independent data.

In **Task 1**, we executed a normal loop where each element of arrays `a` and `b` was added one by one. The CPU handled one addition at a time.

In **Task 2**, we used `#pragma omp simd`, which allowed the compiler to apply SIMD vectorization. This means the processor can add multiple elements of the arrays in a single instruction instead of one at a time.

SIMD works best in situations like this lab because:

- The operation (`c[i] = a[i] + b[i]`) is the same for every element.
- Each iteration is independent.
- The data size is very large (100 million elements).

However, SIMD gives the most noticeable improvement when the program is computation-heavy. If memory access is the bottleneck (as in large array operations), the performance gain may be limited.